

Random Class API

```
public class Random {  
    /** Creates a new random number generator.  Runs in  $\Theta(1)$  time. */  
    public Random() { ... }  
  
    /** Creates a new random number generator using a seed. If two instances of Random are created  
    with the same seed, they will generate and return identical sequences of numbers. Runs in  $\Theta(1)$   
    time. */  
    public Random(long seed) { ... }  
  
    /** Returns the next pseudorandom int from this random number generator's sequence. Runs in  $\Theta(1)$   
    time. */  
    public int nextInt() { ... }  
  
    /** Returns a pseudorandom int value between 0 (inclusive) and the specified bound (exclusive),  
    drawn from this random number generator's sequence. Runs in  $\Theta(1)$  time. */  
    public int nextInt(int bound) { ... }  
  
    /** Returns a pseudorandom int value between origin (inclusive) and the specified bound (exclusive),  
    drawn from this random number generator's sequence. Runs in  $\Theta(1)$  time. */  
    public int nextInt(int origin, int bound) { ... }  
}
```

WeightedQuickUnion Class API

```
public class WeightedQuickUnion {  
    /** Creates a WeightedQuickUnion with n elements numbered 0 (inclusive) to n - 1 (inclusive). */  
    public WeightedQuickUnion(int n) { ... }  
  
    /** Check whether x and y belong to the same set. */  
    public boolean isConnected(int x, int y) { ... }  
  
    /** Union the set that x belongs to and the set that y belongs to. */  
    public void union(int x, int y) { ... }  
}
```

Math Class API

```
public class Math {  
    /** Returns the value of the first argument raised to the power of the second argument.  
    * Runs in  $\Theta(1)$  time. */  
    public static double pow(double a, double b) { ... }  
  
    /** Returns the positive remainder when dividing x by y. Runs in  $\Theta(1)$  time. */  
    public static long floorMod(long x, long y) { ... }  
}
```

PriorityQueue Class API

```
public class PriorityQueue<E extends Comparable<E>> {  
    /** Create a PriorityQueue<E> with elements ordered according to their natural  
    * ordering as defined by the compareTo method of the elements. Runs in  $\Theta(1)$  time. */  
    public PriorityQueue() { ... }  
  
    /** Inserts the specified element into this priority queue. Runs in  $O(\log(N))$  time. */  
    public void add(E e) { ... }  
  
    /** Retrieves and removes the head of this queue, or returns null if this queue is empty. The  
    head of this queue is the element which is smallest according to the compareTo method.  
    * Runs in  $\Theta(\log(N))$  time. */  
    public E poll() { ... }  
  
    /** Determines whether the queue is empty or not. Runs in  $\Theta(1)$  time. */  
    public boolean isEmpty() { ... }  
  
    /** Returns the size of the queue. Runs in  $\Theta(1)$  time. */  
    public int size() { ... }  
  
    /** Retrieves, but does not remove, the head of this queue, or returns null if this queue is  
    * empty. Runs in  $\Theta(1)$  time. */  
    public E peek() { ... }  
}
```

List Interface API

```
public interface List<E> {  
    /** Appends the specified element to the end of this list. Runs in constant time. */  
    void add(E e);  
  
    /** Returns the element at the specified position in this list. */  
    E get(int index);  
  
    /** Replaces the element at the specified position in this list with the specified element. */  
    E set(int index, E element);  
  
    /** Returns the number of elements in this list. */  
    int size();  
  
    /** Returns true if this list contains no elements. */  
    boolean isEmpty();  
  
    /** Lists are defined to be equal if they contain the same elements in the same order. */  
    boolean equals(Object o);  
}
```